

Heisprosjekt rapport

Patrick Vadstensvik Karoliussen, Tristan Mietle

19. mars, 2026. Gruppe 63

I dette prosjektet ble det jobbet med å utvikle ett kontrollsystem til en heis ved hjelp av V-modellen. UML-diagrammer ble laget for å modellere hvordan kontrollsystemet skulle bygges opp, og designet programmet vårt etter diagrammet. Underveis i implementasjonsprosessen møtte vi på noen utfordringer som gjorde at vi måtte endre litt på hvordan vi strukturerte programmet vårt. I denne rapporten skal det reflekteres over hvordan V-modellen og UML har påvirket designprosessen, sluttresultatet, og implementasjon, samt innvirkningen KI har hatt.

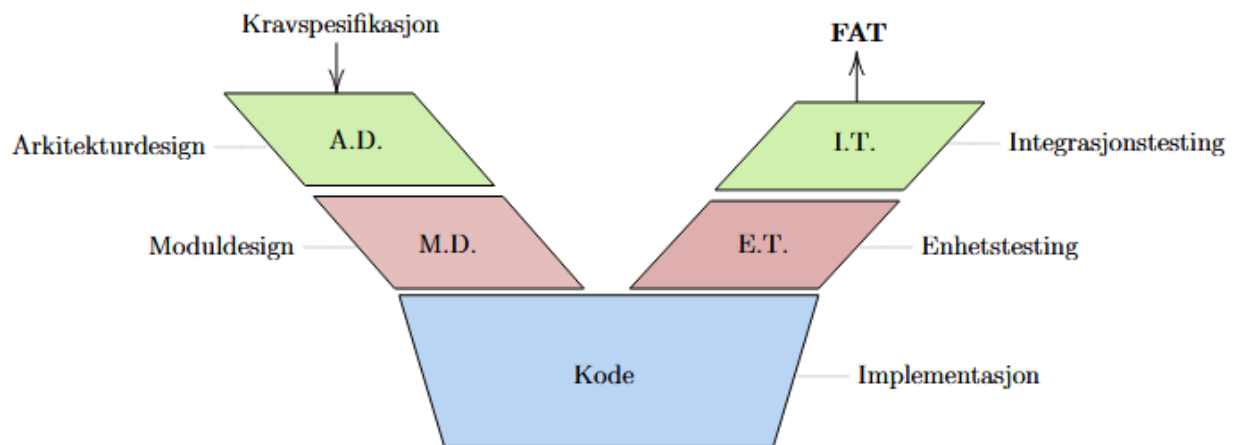


Figure 4: Illustrasjon av V-modellen.

Bruken av den pragmatiske V-modellen og UML

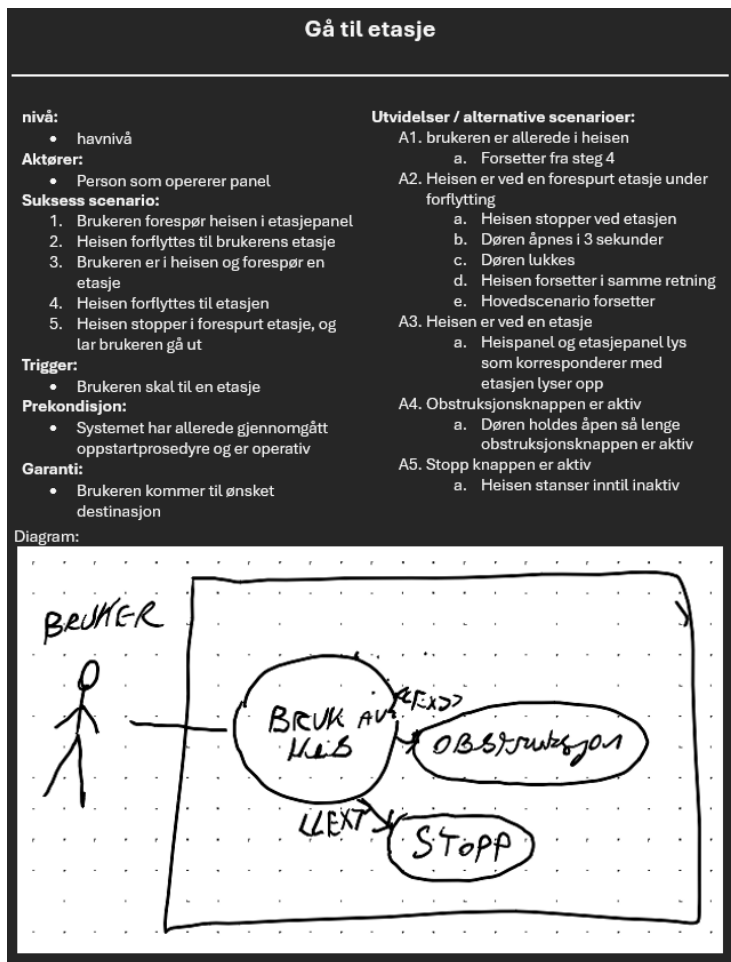
I sin helhet har den pragmatiske V-modellen blitt brukt i prosjektet, ettersom en mer komplisert modell ville gjort organisasjonen av prosjektet mindre produktiv. Denne modellen skal i teorien passe godt til små team på 1 – 4 personer og dermed redusere kostnader, noe den gjorde, men likevel kunne man ha hatt en mer fleksibel variant. Hovedproblemet er at modellen ikke har noen iterativ funksjon, og at man i praksis ender opp med å jobbe iterativt likevel. Derfor kunne man heller ha valgt en versjon av modellen som tillater tilbakekobling i arbeidet, slik at det reflekterer seg i arbeidet. På den andre siden var den pragmatiske V-modellen enkel å bruke, og man fikk en rask organisasjon av

prosjektet. Dette hjalp mye med tanke på at størrelsen på prosjektet er lite, og besparer mye tid.

V-modellen har også påvirket hvordan implementasjonen har blitt gjennomført i ulike faser. Uten en organisasjon modell ville man vanligvis ha hoppet rett inn i koden og skrevet, men for ett prosjekt som krever spesifikasjonskrav har det vært veldig praktisk med en slik modell. Hvis man ikke hadde fått oversikt fra topp først og til bunn sist ville det vært vanskeligere å vite hvordan prosjektet skal implementeres på en produktiv og pen måte. Det har også gjort kodekvaliteten betraktelig bedre. Eneste ulempen var at prosjektet krevde C og ikke C++, noe som gjorde at det ikke ble brukt objektorientert programmering. Mange av diagrammene går ut ifra objekter, men dette ble løst ved å holde oversikten i klassediagrammet.

Modellen har også vært fin for fasene etter implementasjonen, men her ser man også at man i praksis ender opp med en tilbakekoblingsprosess i praksis. Under testfasene finner man gjerne ut at enkelte funksjoner kunne blitt implementert på en annen måte, og man blir dermed nødt til å gå tilbake til implementasjonsfasen for å gjøre endringer. Det samme gjelder bruken av UML diagrammer under implementasjonsfasen.

UML diagrammer har også hatt en ganske stor påvirkning på hvordan koden implementeres og organiseres, mest for forståelse av hvordan spesifikasjonene kan henge sammen med kode. Det ble lagt merke til at de ulike diagrammene har forskjellige ulemper og fordeler for prosjektet, og basert på disse ble de ulike diagrammene valgt:



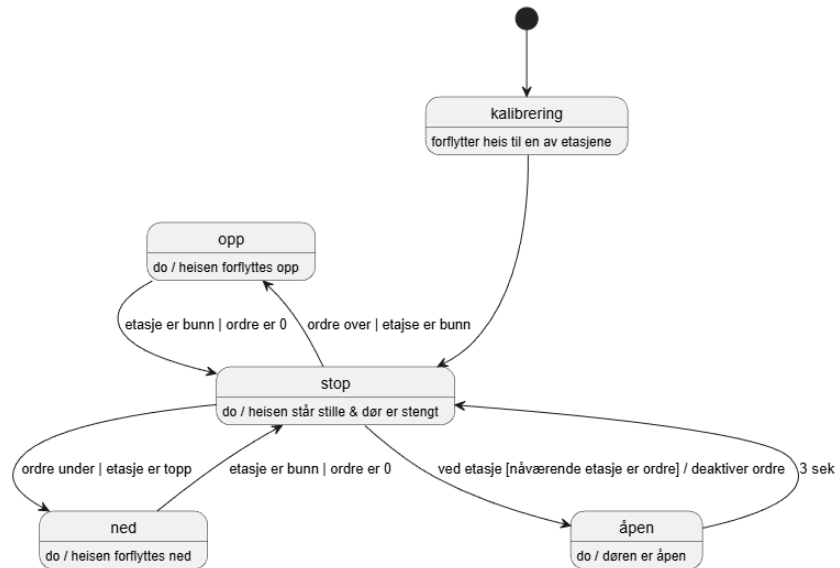
I den første fasen ble det skaffet en oversikt over sammenhengen mellom spesifikasjonskravene og virkeligheten ved å lage ett bruksområde dokument i word. Denne innebærer både ett diagram og tekst om scenarioene som kan komme, samt alternativene man kan komme borti under hovedscenarioet. Dette fungerte godt for å skaffe seg en oversikt over spesifikasjonene og sette dem i en konkret sammenheng. På den andre siden fungerte den ikke godt for å ha noen detaljert forståelse, og heller ikke for oppsettet til programmet som ble laget. Likevel var den verdt å bruke tid på, ettersom den gjør det enklere å lage de andre diagrammene. Det eneste som kan være verdt å kommentere kort på er at selve bruksområde

diagrammet ikke var så nyttig, det var hovedsakelig teksten som ga høyest utbytte på forståelse.

Man kunne også ha laget ett kommunikasjonsdiagram og sekvensdiagram, for å ytterligere øke oversikten og gjøre det konkret. kommunikasjonsdiagram hadde vært fint for å bygge på forståelsen etter bruksområde dokumentet, så lenge man ikke hadde brukt mye tid på det. Man kan også argumentere for at det beste er å hoppe rett på sekvensdiagrammet. Sekvensdiagrammet inneholder mye av den samme informasjonen som man har i kommunikasjonsdiagrammet, så alternativt kunne man hoppet rett på denne, og så gå over til andre fase.

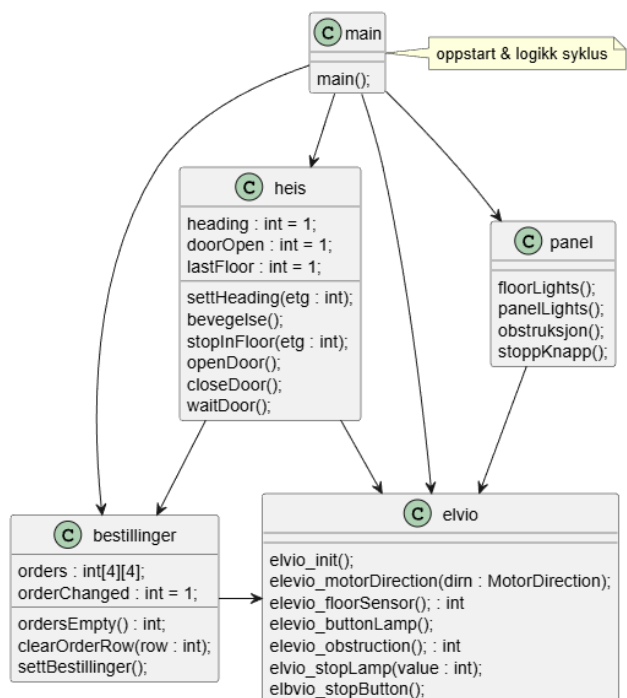
I den andre fasen ble det brukt tilstand diagrammer og klassediagram, disse hadde høyest utbytte i V-modellen, sammen med implementasjonen. Tilstands diagrammet var fint for å

få forståelse av
virkningen til selve
heisen, og hvordan den
kan programmeres. Å se
tilstands-perspektivet til
systemet gjorde det
enklere å fikse bugs og ha
en overordnet logikk på
kontrollsystemet. På den
andre siden kunne den
ikke representere
logikken vi endte opp
med perfekt fordi mye av



tilstandsmodellen er basert på rene kondisjoner. Det gjorde at vi heller baserte oss mest på
klassediagrammet. Tilstands diagrammet kunne i teorien ha blitt brukt bedre, spesielt
sammen med klassediagrammet, men likevel har det gitt en forståelse av hvordan
kontrollsystemet kan representere ulike tilstander som kontrollerer heisen. Det gjorde at
man kunne dele opp i hovedsakelig 4 tilstander.

Klassediagrammet passet veldig godt med oppgaven og var enkel å komme seg inn i pga
tidligere erfaringer med objektorientert programmering. Diagrammet gjorde det enkelt å
holde oversikt over funksjonene systemet innebærer, og hvordan de kan organsieres i
grupper. På den andre siden var den ikke god på kommunikasjonsdelen eller logikkdelen av



systemet, men disse er allerede løst via
tilstandsdiagrammet. En ting som kan
være verdt å kommentere på er at man
hadde fått enda større utbytte av
klassediagrammet hvis man også hadde
laget sekvensdiagrammet. Det er fordi
sekvensdiagrammet og klassediagrammet
hadde vært gode komplement, nettopp
fordi fordelene og ulempene jevner
hverandre ut. Klassediagrammet var også
det diagrammet som ble brukt mest, og
endret mest på. Endringene på
klassediagrammet skjedde under
implementasjonsfasen, nettopp fordi man

finner ut at det finnes bedre måter å organisere koden på.

Om bruk av UML & V-modell har gjort det enklere, og alternative metoder

I sin helhet har V-modellen og UML diagrammene gjort det enklere å lage ett godt system, men ikke nødvendigvis raskere eller enklere å lage systemet. Prosjektet er såpass lite at man kunne vært mer effektiv ved å gå rett inn på implementasjonsfasen, men da hadde man ikke fått samme kodekvalitet. Dermed kan man si at modellene er gode metoder for å øke kvaliteten for små prosjekter, på bekostning av litt tid. I programmering er alt avhengig av perspektiv. Hvis prosjektet hadde vært større hadde det nesten vært nødvendig med bruk av modeller, spesielt hvis man er mange på prosjektet. Da hadde man både økt produktiviteten og kvaliteten. Og som tidligere nevnt ble det en del hopping imellom de ulike fasene, derfor kunne man kanskje ha brukt en alternativ modell som for eksempel smidig (agile) programvareutviklingsmodeller.

Hva KI har blitt brukt til i prosjektet og andre poeng

Når det gjelder kunstig intelligens har det ikke blitt brukt til å generere store deler av koden vår, og har hovedsakelig brukt det til å hente informasjon når det var noe vi ikke forsto. Det ene tilfellet vi brukte KI til å generere kode var for å finne ut av hvordan vi skulle bruke time-biblioteket for å implementere waitDoor-funksjonen vår, siden ingen av oss visste hvordan man skulle bruke time-biblioteket. Utover det har vi brukt KI til forslag til hvordan man kan gå frem til å løse en oppgave.

Vi har gjort det meste av arbeidet selv, og tror derfor ikke at KI har hatt en veldig stor effekt på arbeidet vårt. Vi lagde strukturen, nesten all koden og alle diagrammene selv, og vi har hovedsakelig brukt KI til å finne ut av hvordan det kan implementeres ideer vi allerede har tenkt ut selv.

Når det gjelder robusthet, skalarbarhet og vedlikehold har ikke oppgaven nevnt noen eksplisitt prioritering av dette, men har bruken av UML og V-modellen bidratt til at noen av disse egenskapene har blitt fremmet.

Vi har designet kontrollsystemet slik at det lett kan utvides til flere etasjer, og det skal (i teorien) kunne funke på en heis med evig mange etasjer. Koden er ikke designet for at den skal kunne betjene flere heiser samtidig, og må nok endres en del før den kan brukes som en del av et mer komplisert system.

Bruken av UML-diagram og V-modellen ga oppgaven og designfasen en god struktur, som gjorde det mye lettere å gjøre oppsettet til kontrolleren oversiktlig. Hvis vi ikke hadde brukt

noen av modellene hadde vi sannsynligvis brukt litt mindre tid på planleggingen av koden, og endt opp med enda mer spagettikode og litt færre funksjoner. Modellene har derfor gjort det lettere å planlegge hvordan vi skulle strukturere programmet, og har derfor, selv om det ikke var en sentral del av oppgaven, gjort at programmene er mer robuste og skalerbare.

Avslutningsvis er det verdt å nevne at vi synes UML-diagrammet fungerte veldig bra for å få oversikt over hvordan vi ville strukturere koden, og hvordan vi ville at programmet skulle se ut. Selv om vi gikk bort fra det opprinnelige UML-diagrammet fikk vi likevel ganske mye ut av å lage diagrammet, og det siste diagrammet vårt er tross alt ganske likt det første.

Vi fulgte ikke V-modellen til punkt og prikke, og måtte gå tilbake til planleggingsfasen en god del ganger. Selv om vi ikke fulgte den veldig nøye fikk vi mye ut av å bruke den, og den gjorde at koden vår ble oversiktlig og strukturert.